IceEdge: Thermal-Aware Machine Learning Inference Serving for Emerging Edge Applications

OMAIS SHAFI*, Department of Computer Science and Engineering
Indian Institute of Technology, New Delhi, India
MOHAMMAD KHALID PANDIT, Department of Computer Science and Engineering
National Institute of Technology, Hamirpur, India
ARPAN GUJARATI, Department of Computer Science
University of British Columbia , Canada
AASTHA MEHTA, Department of Computer Science
University of British Columbia , Canada
RIJUREKHA SEN, Department of Computer Science and Engineering
Indian Institute of Technology, New Delhi, India

Smart city applications require machine learning (ML) capabilities at the edge, for which they require edge devices equipped with specialized processors (e.g., GPUs). However, deploying such high-end edge devices in large numbers is challenging, particularly in regions that face infrastructure and budget constraints (e.g., cities lacking ubiquitous broadband) or extreme weather conditions (e.g., hot temperatures over sustained periods of time). Hence, custom ML frameworks are necessary that can support efficient and safe consolidation of multiple applications with diverse workloads on fewer edge servers. We present IceEdge, an inference serving framework for CPU-GPU based edge computing servers, which isolates colocated applications using existing container mechanisms and provides a deadline- and thermal-aware scheduler for the ML workloads. We implement IceEdge for a Dockerized environment on NVIDIA Jetson Xavier AGX platform. IceEdge outperforms state-of-the-art real-time inference serving frameworks Triton and DeepRT, improving the frame drop rate by up to 15% and reducing the temperature rise by around 12°C compared to Triton inference serving framework.

CCS Concepts: • **Computer systems organization → Embedded systems**; • **Neural Networks → Optimizing inference**; • **Scheduling → Thermal Control**.

Additional Key Words and Phrases: Neural network inference server, concurrency, scheduling, thermal control

*Most of this work was done while the author was at the Indian Institute of Technology Delhi. The author is currently with the ARM Cambridge, UK.

Authors' addresses: Omais Shafi, Department of Computer Science and Engineering
Indian Institute of Technology, New Delhi, India, omais.shafi@cse.iitd.ac.in; Mohammad Khalid Pandit, Department of Computer Science and Engineering
National Institute of Technology, Hamirpur, India, khalid@cse.iitd.ac.in; Arpan Gujarati, Department of Computer Science
University of British Columbia , Canada, arpanbg@cs.ubc.ca; Aastha Mehta, Department of Computer Science
University of British Columbia , Canada, aasthakm@cs.ubc.ca; Rijurekha Sen, Department of Computer Science and Engineering
Indian Institute of Technology, New Delhi, India, riju@cse.iitd.ac.in.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than ACM must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
© 2026 Association for Computing Machinery.
XXXX-XXXX/2026/3-ART $15.00
https://doi.org/10.1145/nnnnnnn.nnnnnnn

ACM Reference Format:
Omais Shafi, Mohammad Khalid Pandit, Arpan Gujarati, Aastha Mehta, and Rijurekha Sen. 2026. IceEdge: Thermal-Aware Machine Learning Inference Serving for Emerging Edge Applications. 1, 1 (March 2026), 26 pages. https://doi.org/10.1145/nnnnnnn.nnnnnnn

1 INTRODUCTION

A core tenet in the design and development of "smart cities" is the use of digital platforms to monitor and maintain public infrastructure, to improve the quality of life and safety of citizens [21]. Rapid growth in sensing and edge computing hardware as well as AI-ML models for fast and accurate processing of sensor data are key enablers. For example, health officials can be notified in advance if a neighborhood has dangerous particulates in the air [14] and traffic rule violations can be automatically reported before they become a menace [20, 40]. Many companies are exploring the use of state-of-the-art reinforcement learning methods for real-time management of traffic congestion [11–13, 42, 43].

The recent developments in ML research has led to deployment of such powerful models on resource constrained edge platforms [30, 31, 37]. Due to low latency and real time requirements applications have to maintain certain Quality of service (latency frame drop rate etc) akin to specific tasks. Modern edge computing devices with specialized processors (e.g., GPUs), such as NVIDIA Jetson devices, have now made it possible to run low-latency ML inference even at the edge. However, deploying such devices in large numbers is challenging. The procurement and installation costs for such devices can comfortably exceed 500 USD per unit. In regions that face infrastructure and budget constraints, this is hugely detrimental, particularly if each individual smart city application requires separate dedicated edge computing platforms (e.g., for proprietary or security reasons).

One of the most important *(often overlooked)* aspect of executing ML applications on edge devices is that it often leads to rise in platform temperature as they are not equipped with cooling facilities like in a data-center; and ML applications being compute-heavy push these platforms to their limits. Thermal control in the OS is oblivious to application needs, e.g., Linux's DVFS controller may reboot or de-throttle the device, forcing the application to be unavailable or use a low-precision model at a critical time. On the other hand, switching off the OS thermal control is not recommended as it affects the platform reliability.

While several inference serving frameworks have been proposed for the cloud platforms [16, 22, 33], these frameworks cannot be directly deployed on the edge as they are optimized for powerful server machines and prioritize throughput over real-time guarantees. The inference serving frameworks designed for edge devices [18, 23, 45, 46, 49] have ignored isolation between multiple tenants. NVIDIA's Triton Inference Server [5] supports container-based isolation for applications, but it does not allow applications to configure their predictability requirements. Moreover, these frameworks ignore the thermal aspects of inference serving.

This work. We present *IceEdge*, thermal-aware inference serving framework for CPU-GPU based edge computing platforms that addresses the aforementioned challenges. IceEdge supports *multi-tenancy* so that multiple applications can share an edge platform. IceEdge isolates co-located applications using existing container mechanisms. Application requests (e.g., raw inputs from sensors) are forwarded directly from the OS to the container, and then executed on the GPU. A central, lightweight user-space controller manages the schedule of all ML inference tasks on the GPU. The controller receives only request metadata from the OS, makes *deadline-aware global scheduling* decisions, and then conveys these to the respective containers. As many edge applications are soft real-time (where missing a deadline occasionally does not cause catastrophic failure), IceEdge's scheduling algorithm uses *2-way concurrency* (running two models concurrently for execution)

that allows consolidation with good GPU utilization but without sacrificing each application's quality-of-service. IceEdge is also *thermal-aware*. It does not rely on OS thermal control; instead, its controller carefully controls the CPU-GPU clock frequencies considering both the dynamic workload and the thermal safety constraints of the edge platform.

In summary, *IceEdge is the first deadline-, concurrency-, and thermal-aware ML inference serving framework for the edge* to the best of our knowledge. We implement IceEdge in C++ for a Dockerized environment on the NVIDIA Jetson Xavier AGX platform [1]. We evaluate our implementation against NVIDIA's Triton Inference Server [5] (which comes along with the Jetson SDK and widely used in practice) and another baseline derived from DeepRT [45] (a state-of-the-art soft real-time inference serving framework), using both synthetic object detection workloads and production workloads derived from an urban traffic management system [35]. IceEdge outperforms both baselines in terms of throughput (frames dropped for video workloads) and thermal control (temperature profile). IceEdge reduces the frame drop rate by around 15% and 8% respectively compared to Triton and DeepRT while reducing the temperature rise by around 12°C and 10°C respectively.

2 ICEEDGE DESIGN CHOICES

We elaborate on IceEdge's key design goals and motivate our design choices through empirical results. We motivate and discuss our scheduling approach directly in Section 3.

Containerized environment for multi-tenancy. Consider three applications running on an edge device deployed at a traffic intersection: (a) traffic rule violation detection, (b) traffic density measurement and signal control to minimize congestion, and (c) curbside parking space management. Each of these applications would use state-of-the-art computer vision models for object detection (e.g. to detect vehicles, number-plates, empty parking spaces) or for image classification (e.g. to read the characters in number-plates). These computer vision models might have different architectures and pre-trained weights, which are all proprietary information for the companies deploying each application. Since the edge devices need to host multiple applications from potentially mutually-distrusting vendors, IceEdge should isolate the applications of different vendors to ensure confidentiality and integrity of the ML models and data, and to ensure fault isolation between the tenants.

Applications on the edge may be subject to several threats, such as platform compromise [17], and direct or side-channel attacks from compromised co-tenants [24]. Mitigations [9] for such threats require hardware support [7, 8, 27], which is not yet ubiquitous on edge CPU-GPU devices [38, 41]. Therefore, implementing strong isolation mechanisms while retaining predictable performance characteristics for real-time inference serving workloads is a non-trivial challenge and beyond the scope of this work.

IceEdge relies on light-weight isolation across applications using containers. Our design choice is inline with the state-of-the-art supported by industry-grade platforms, such as Triton [5], and is reasonable because vendors are interested in preserving their own confidentiality and integrity, and not attacking other tenants, which might lead to legal repercussions. Each application vendor instantiates one or more Docker containers on the shared edge platform, and the total number of containers are limited by the platform resources, i.e., CPU cores, RAM, and disk space.

Per-model batching and 2-way concurrency. Several prior frameworks have demonstrated the efficacy of batching inference requests in improving inference throughput [5, 33, 45]. Given the isolation constraints of co-located applications, IceEdge utilizes batching on a per-model basis, i.e., it only batches inference requests of a single tenant model.

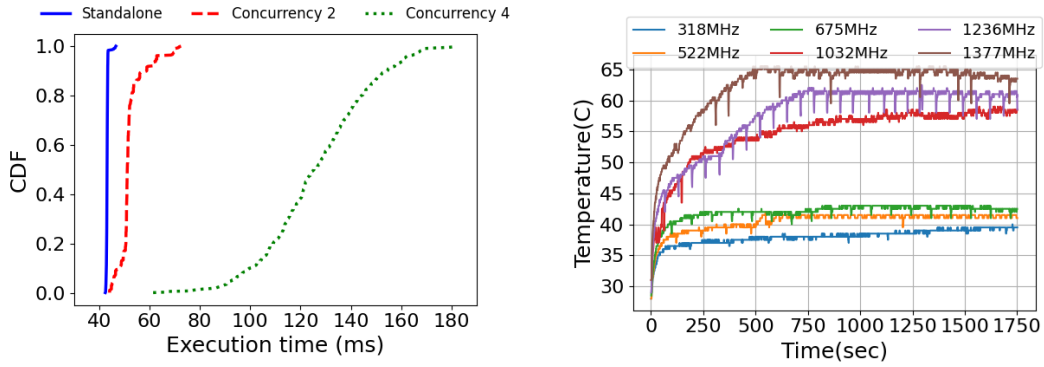
On the other hand, when applications attempt to schedule multiple models on GPUs concurrently, the CUDA framework schedules the model kernels in a time-sliced manner [29]. This means that the execution time of one model is affected by the size, number of layers, and execution time of

the layers of other models that are executing concurrently, which increases the variance in the model execution times, sometimes, by 100x [22]. Consequently, prior work [22, 45] has eschewed concurrent execution of ML models.

Cloud-based models tend to utilize a large portion of the GPUs during their execution. Moreover, cloud platforms can scale out GPUs to meet their workload demands. Therefore, cloud-based inference serving frameworks can afford to not rely on concurrency for improving inference throughput. However, as edge GPU resources are limited, we re-examine this choice of whether to completely ignore concurrency mechanisms in IceEdge or not.

We run multiple concurrent instances of the ResNet-18 model on NVIDIA Jetson AGX platform with vanilla CUDA 10.1 framework. For each model, we use an input stream consisting of a camera video feed. In each stream, frames arrive on the CPU of the platform at an average rate of 30fps, which are sent in fixed-sized batches of 12 frames to the respective model instance on the GPU for inference.

We run experiments for three concurrency configurations: 1 (standalone), 2, and 4. In each case, we measure the per-batch inference time and total throughput across all concurrent model executions. Figure 1a shows the CDF of the inference times for the three concurrency configurations. We observe that at concurrency 2, the mean and 95th percentile latencies increase by 21% and 44.6% compared to standalone, however we observed that the inference throughput for concurrency 2 is 2x the standalone throughput in our experiments. We only show the CDF due to lack of space. This shows that 2-way concurrency can help improve inference throughput while causing acceptable impact on latency predictability. Thus, *IceEdge leverages 2-way concurrency while scheduling inference requests from multiple applications.*



(a) (b)

Fig. 1. (a) Inference latency (b) GPU frequency vs. Temperature

Thermal control. While cloud datacenters are equipped with sophisticated power management and cooling systems, edge devices may be deployed in environments with fluctuating ambient temperatures and limited or no cooling capacity. We perform two experiments on NVIDIA Jetson AGX to understand the effects of GPU operating frequencies on GPU temperatures and workload performance. First, Figure 1b shows the effect on GPU temperatures over time when the GPU operating frequency is configured at different values.

Second, we study the effect of varying GPU frequencies on application performance. We run an InceptionV3 model on the same platform with input consisting of a video stream. Frames arrive on the CPU platform at an average rate of 30fps, which are sent to the GPU for inference. We

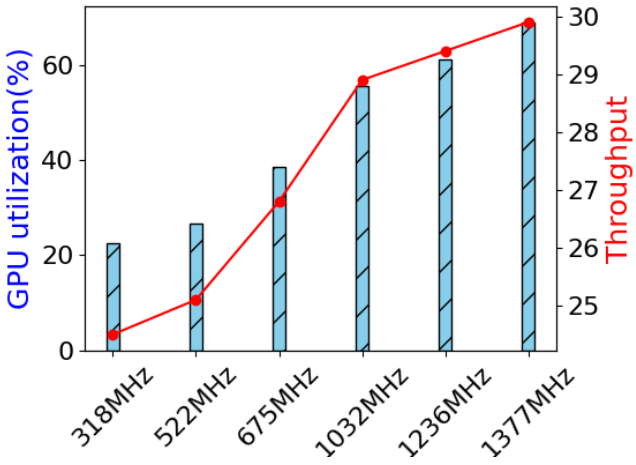


Fig. 2. Throughput and GPU Utilization

set the GPU operating frequency at fixed levels between 318 MHz and 1377 MHz, as supported by the platform. Figure 2 shows the inference throughput and GPU utilization at different GPU frequencies. Throughput and GPU utilization are proportional to GPU frequencies, which in turn affect the GPU temperatures.

In response to rise in platform temperatures, dynamic voltage and frequency scaling (DVFS) mechanisms on a platform scale down clock frequencies, which would slowdown inference serving applications. In more drastic scenarios, DVFS may even cause the platform to reboot, leading to longer periods of unavailability.

For predictable inference serving, IceEdge takes platform temperature into account while scheduling inference requests to avoid thermal shutdowns and enhance platform lifetime.

3 ICEEDGE SYSTEM DESIGN

Figure 3 illustrates an overview of IceEdge's architecture on a single edge device. A device hosts multiple Docker containers, where each container contains one pre-trained model that serves a specific client. A client, such as a sensor or a camera, provides inputs (e.g., video frames) that are forwarded to the respective Docker container and queued in an inference queue within the container. A central controller runs natively on the device, which has a global view of system state, including all the containers and their requests, as it regularly receives metadata regarding each request from the containers over inter-processor communication. This allows isolation of models among the containers, and at the same time allows the controller to come up with a global schedule. A key idea that differentiates IceEdge from prior work is that IceEdge allows limited concurrency (up to a factor of two in this case) but explicitly controls how and when concurrent executions are allowed. To achieve this, IceEdge relies on a profiling table, that informs its controller how inference times vary when any two specific models or model batches are executed concurrently on the GPU. Specifically, the profiling table starts with inference times profiled statically (in case of concurrent execution, this is simply a sum of the two individual inference times), and is regularly updated using measurements from more recent inferences. Such a table-driven approach may seem intractable in cloud settings where thousands of different clients with different model types interact with the server. However, this approach is manageable at edge, as the number of distinct clients and models is small (e.g., under ten).

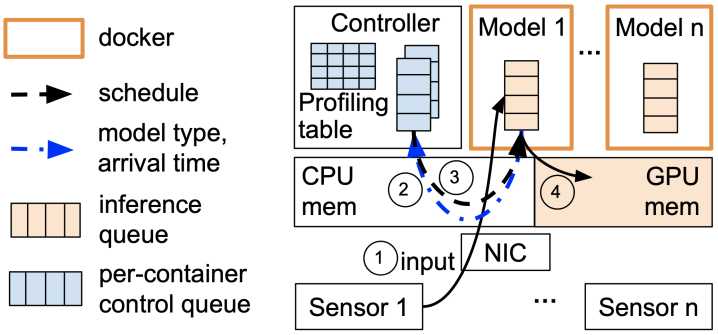


Fig. 3. IceEdge design and workflow. ① Sensor inputs are sent to a model in a Docker container. ② The container forwards request metadata to a central controller. ③ The controller determines the schedule for requests and sends the schedule to respective containers. ④ The container forwards the requests to the GPU for inference as per schedule.

Once IceEdge has the knowledge of all pending requests and inference profiles, it evaluates several schedule templates, starting with the simplest one based on the earliest deadline first (EDF) algorithm and without considering any concurrency, and then refining the template iteratively by first improving the average *batching factor*, and then the average *concurrency factor* across all requests. IceEdge assumes that requests from each client are assigned a fixed client-specific relative deadline. Absolute deadlines can then be derived from the arrival time of the request and the relative deadline. Once the controller is satisfied with a scheduling template (i.e., it cannot improve the template any further), it splits the template into partial application-specific schedules and sends the partial schedules to the respective Docker containers. Each application is then responsible for executing the specific requests at the specific time as indicated by the partial schedules.

The IceEdge controller also features a thermal control module that uses the number of pending requests as an input and accordingly auto-scales the GPU frequency up or down. We describe the controller design in detail below.

3.1 Scheduling

Let $M = \{M_1, M_2, \ldots M_m\}$ denote the set of m distinct models. Each Docker container is responsible for one application model, but it may maintain multiple instances of the same model with different configurations for request batch sizes. We use B distinct batch sizes, $B = \{B_1, B_2, \ldots, B_b\}$ for each model. Hence, we let each model M_i denote the set of b different model instances as $M_i = \{M_{i,B_1}, M_{i,B_2}, \ldots, M_{i,B_b}\}$, where M_{i,B_k} denotes the model instance that accepts inputs in batches of size k. Each model M_i receives an infinite stream of inference requests $R_i = \{R_{i,1}, R_{i,2}, R_{i,3}, \ldots\}$. Each request $R_{i,k}$ is characterized by its arrival time $a_{i,k}$, absolute deadline $d_{i,k}$, and finishing time $f_{i,k}$. Upon arrival, the request is first queued, then processed so that it matches the input format required by the model, then queued again, and finally dispatched to the GPU for inference.

Formally, we denote a partial schedule using an ordered set of computations, where each computation is defined by a tuple consisting of its starting time, the batch size to be used, the set of requests in the batch, and an absolute deadline, *e.g.*,

$$\begin{aligned} S_{\textit{eg-GPU-sched}} = \{&(t_1, B_1, \{R_{i,1}\}, d_{i,1}),\\ &(t_2, B_4, \{R_{i,2}, R_{i,3}, R_{i,4}\}, \min(d_{i,2}, d_{i,3}, d_{i,4})),\\ &(t_3, B_2, \{R_{i,5}, R_{i,6}\}, \min(d_{i,5}, d_{i,6}))\}. \end{aligned} \tag{1}$$

The number of requests in each computation must not exceed the batch size and the deadline for the batch is chosen as minimum among all the request deadlines. Note that so far we have not discussed anything about concurrency at all.

Let D_{GPU} denote the dataset (realized using a profiling table) of model inference times under different scenarios. Suppose the controller is invoked at time t_{now}. Suppose that, as per the scheduling done in the past, the GPU becomes free at some point $t_{free} > t_{now}$ in the future. The controller maintains a local state to keep track of t_{free}, which is updated for every iteration of the scheduling interval by an acknowledgement of the completion of the execution from the containers. Our controller algorithm follows some steps before sending the partial schedules to the individual containers.

Step 1: Earliest Deadline First (EDF). We start with EDF as it is optimal for uniprocessor scheduling [10, (Ch.4)] and provides a convenient starting template. In this case, we have a single GPU where all inference requests will be executed, akin to a uniprocessor system. However, GPU inference requests are slightly different from traditional uniprocessor real-time systems in the sense that they can be both non-preemptive (if no other inference request is schedule concurrently) or partially preemptive (e.g., if two inference requests are scheduled concurrently, their individual kernels execute non-preemptively but multiple kernel executions may interleave).

Let S_{EDF} denote the starting template that schedules all requests in $\cup_{i=1}^{m} R_i^{t_{now}}$ using EDF, starting at time t_{free}. None of the requests in S_{EDF} are batched yet. Therefore, the schedule S_{EDF} basically resembles the following:

$$\begin{aligned} S_{EDF} = \{&(t_{free}, B_1, \{R_{x_1,y_1}\}, d_{x_1,y_1}),\\ &(t_{free} + \Delta_{x_1}, B_1, \{R_{x_2,y_2}\}, d_{x_2,y_2}),\\ &(t_{free} + \Delta_{x_1} + \Delta_{x_2}, B_1, \{R_{x_3,y_3}\}, d_{x_3,y_3}),\\ &\ldots\}, \end{aligned}$$

where Δ_{x_1} is the worst-case execution time when executing request R_{x_1,y_1} in isolation (as derived from profiling dataset D_{GPU}). Δ_{x_2} is similarly defined.

Step 2: Batching. The next stage is iteratively improving S_{EDF} to reduce its makespan while maintaining the timeliness of all requests. The goal of this step is to batch as many requests as possible.

The batching algorithm is shown in Algorithm 1. Consider the following schedule showing two consecutive requests of a model M_{x_1}. (Note that there might be requests of other models between these two requests.)

$$\begin{aligned} S'_{EDF} = \{\ldots \quad &(t_1, B_1, \{R_{x_1,y_1}\}, d_{x_1,y_1}),\\ &\ldots\\ &(t_2, B_1, \{R_{x_1,y_2}\}, d_{x_1,y_2}), \quad \ldots\} \end{aligned}$$

In this schedule, suppose that R_{x_1,y_2} is expected to miss its deadline. This is possible if $t_2 + \Delta_{x_1} > d_{x_1,y_2}$. However, we could batch R_{x_1,y_1} and R_{x_1,y_2} if the following criteria are satisfied: **(a)** batching should actually help R_{x_1,y_2} satisfy its deadline, and **(b)** batching should not end up causing a deadline miss for the first request R_{x_1,y_1}. **(c)** requests that were earlier going to be executed between R_{x_1,y_1} and R_{x_1,y_2} will now be executed after the batch of R_{x_1,y_1} and R_{x_1,y_2}; hence, their starting times will need to be pushed back, which should not cause them to miss their deadlines. This is the condition implemented in IS_BATCHABLE in Line 7. IS_BATCHABLE checks whether two batches of requests from the same model can be merged. The function ensures that (a) the later request meets its deadline when batched, (b) the earlier request does not miss its own deadline, and (c) rescheduling requests that fall between the two does not cause them to miss deadlines.

Algorithm 1 Batching Algorithm

Input: S_{EDF} ▷ Request list ordered by deadlines
Output: S'_{EDF} ▷ Schedule with batched requests

```
1:  function SCHED_BATCH( )
2:      while S'_EDF ≠ S_EDF do                          ▷ repeat until S_EDF fixed
3:          S'_EDF ← S_EDF
4:          for all b ∈ [S_EDF.first, S_EDF.last] do
5:              ▷ find next batch in S_EDF for the same model
6:              next_b ← get_next_entry(model(b), S_EDF)
7:              if IS_BATCHABLE(b, next_b, S'_EDF) then
8:                  ▷ move requests from next_b to b
9:                  b ← coalesce(b, next_b)
10:                 ▷ update start times, deadlines of requests
11:                 for all req ∈ b do
12:                     update_params(req)
13:                 ▷ move start times of interim batches back
14:                 for all iter_b ∈ [b, next_b] do
15:                     update_params(iter_b)
16:                 ▷ move start times of later batches forward
17:                 for all iter_b ∈ [next_b, S_EDF.last] do
18:                     fixhole(iter_b)
19:     return S'_EDF
```

If the conditions are satisfied, we coalesce the requests (Line 9) into a single batch. Line 9 merges the request sets into a single batch, sets the batch deadline to the minimum deadline of all merged requests, and looks up the new worst-case execution time from the profiling dataset D_{GPU}. We then update the start times and deadlines for the requests in the batch based on the worst-case execution time of the batch (Line 11).

Next, we push back the start times for the requests (or batches) between R_{x_1,y_1} and R_{x_1,y_2} based on the worst-case execution time of the batch (Line 15). Finally, we move the start times for the requests (or batches) *after* R_{x_1,y_2} up by diff $= 2 * \Delta_{x_1,1} - \Delta_{x_1,2}$ (Line 18). $\Delta_{x_1,1}$ is the worst-case execution time (WCET) of a single request of model M_{x1} when executed in isolation (batch size = 1) whereas $\Delta_{x_1,2}$ is the worst-case execution time when two requests of model M_{x1} are executed together in a batch of size 2. It basically eliminates idle periods ("holes") introduced by batching. It moves forward the start times of subsequent requests so that GPU utilization remains tight and no slack time is wasted.

We apply the algorithm over the entire schedule S_{EDF} repeatedly until we reach a fixed point. Note that we may even end up with batches of sizes greater than two, *e.g.*, when we batch a request with another batched request. We denote the final schedule at the end of this step as S'_{EDF}.

Step 3. Allow concurrency. Next, the controller looks for opportunities for concurrent scheduling of request batches in pairs of models. Suppose the schedule at the end of the current step is $S''_{EDF} = \{E_1, E_2, \ldots, E_n\}$, consisting of n model instances ready for GPU execution. IceEdge begins by pairing two ML models M_i and M_j for possible concurrent execution subject to avoidance of deadline misses. In other words, the choice of concurrent models is made if their execution finishes before the start of next pair of models. Note that IceEdge initially assumes the run-time of concurrent models M_i and M_j to be the summation of their individual solo run times as recorded in the profiling table. However, IceEdge records the execution time of batched concurrent executions in a profiling table. After some finite number of iterations, IceEdge updates the concurrent inference time for all batch configurations in the profiling table, and checks for new potential concurrent executions. We denote the final schedule at the end of this step as S''_{EDF}.

Algorithm 2 IceEdge Concurrent Scheduling Algorithm

Input: S'_{EDF} ▷ Schedule at the end of Algorithm 1
Output: S''_{EDF} ▷ Schedule with concurrent batch pairs

function SCHED_CONCURRENT( )
  for all $b_i, b_j \in S'_{EDF}$, model(b_i) ≠ model(b_j) **do**
    if end_time($b_i || b_j$) ≤ min_deadline(b_i, b_j) **then**
      for all $req \in b_i, req' \in b_j$ **do**
        ▷ update start times, deadlines for batches
        update_params(req)
        update_params(req')
      ▷ update start times of other batches
      fixhole( )
  $S''_{EDF} \leftarrow S'_{EDF}$ ▷ schedule updated in place
  return S''_{EDF}

Step 4. Splitting partial schedules. In the final step, the controller simply divides the generated schedule into model-specific schedules and dispatches each schedule to the respective container. The controller additionally modifies t_{free}, i.e., when the GPU is anticipated to become idle once more, based on the final schedule. However, it will still wait for the final acknowledgement from the container with the execution time and update the t_{free} accordingly.

In addition, IceEdge also uses few policies to decide whether the inference request should be sent to the GPU for execution. If the GPU execution time of the current request is greater than the next scheduling interval, it means we cannot schedule the next execution to the GPU till the current request completes. In this case, IceEdge checks if the current GPU execution time is greater than 50% of the scheduling interval time, it then skips that iteration of the controller invocation. Moreover, the containers also maintain a local state and drops the requests that cannot meet the deadlines.

3.2 Thermal Control

One of the key aspects of IceEdge's design is its thermal controller, which ensures thermal safety while balancing application requirements. Prior systems [5, 45] use dynamic voltage and frequency scaling (DVFS) to control temperature increase. However, DVFS-based solutions impact the performance of the system [35].

Given the insights from Section 2, IceEdge's thermal control takes both the system temperature and application performance requirements into account while determining the GPU frequency levels to switch to.

We define F_{turbo} as the maximum (turbo) frequency of the device and F_{safe} as the desired GPU frequency which maintains the trade-off between performance and temperature. The choice of F_{safe} is made empirically and varies among the devices. DVFS-based inference systems are found to reach F_{turbo} for even moderate workloads. IceEdge implements a proactive thermal control which executes the workload at F_{safe} and only switches to F_{turbo} when the performance requirement is high, akin to the presence of burst in the input frame rate in our setup.

Initially, the system frequency is set to F_{safe} prior to running THERMAL_CONTROL (Algorithm 3). The algorithm is invoked at each controller epoch and makes frequency switching decision based on the input queue length (input rate), T_{queue} and rate of change in the system temperature. T_{queue} is the minimum queue length that helps the IceEdge controller to make frequency switching decisions based on input load. For all input queue lengths less than T_{queue}, IceEdge maintains the GPU frequency at F_{safe}, otherwise switches to F_{turbo} in-order to maintain the performance. IceEdge

Algorithm 3 IceEdge Thermal Control Algorithm

Input: $S''_{EDF}, F_{now}, Temp_{now}$ ▷ Schedule from Algorithm 2; current system frequency, temperature

1: **function** THERMAL_CONTROL( )
2: $T_{queue} \leftarrow S''_{EDF}.len$
3: **if** $T_{queue} \geq 12$ **then**
4: $F_{now} \leftarrow F_{turbo}$ ▷ Switch to turbo frequency
5: $Temp_{switch} \leftarrow Temp_{now}$ ▷ system temp. at the time of switch.
6: **if** $iter \mod M = 0$ **then** ▷ Execute the function after every M = 60 seconds
7: THERMAL_SWITCH($Temp_{now}$,$Temp_{switch}$)
8: **function** THERMAL_SWITCH($Temp_{now}$,$Temp_{switch}$)
9: **if** $Temp_{now} - Temp_{switch} \geq 5$°C **then**
10: $F_{now} \leftarrow F_{safe}$ ▷ switch back to F_{safe}
11: **else**
12: $F_{now} \leftarrow F_{turbo}$ ▷ continue at turbo frequency
13: $Temp_{switch} \leftarrow Temp_{now}$ ▷ update the temperature

maintains the turbo frequency for time interval M seconds which avoids the frequent switching in-case of periodicity in input bursts. In case of constant high performance input patterns, IceEdge also keeps track of change in temperature and performs proactive measures to avoid the sharp rise in temperature. If the rise in temperature in the last M seconds is more than T_{temp} degrees (as shown in THERMAL_SWITCH function), IceEdge switches back from F_{turbo} to F_{safe} until the temperature is reduced by T_{temp}.

4 IMPLEMENTATION

We implement IceEdge in C++ and deploy it on NVIDIA Jetson AGX [1], which consists of 8 CPUs and one Volta GPU with 512 CUDA cores, and runs Ubuntu 18.04 OS. We use TVM v0.9 and LLVM v13.0 for hardware-specific tuning and optimization [15], and CUDA 10.1 for GPU inference. The overhead of running IceEdge controller is roughly around 13ms consisting of the computations needed to perform EDF, batching and checking for concurrency.

Our setup consists of multiple Docker containers, up to one per core. Application vendors launch one or more containers. Each container hosts a particular DNN model, with different executables tuned with TVM for different batch sizes, i.e., 1, 4, 8, and 12. This allows us to maximize the effect of batching, at the cost of some space complexity, as the latency for inferring a batch of size b (using a model tuned and optimized for this batch size) is typically less than b times the latency for inferring a single request [22]. All tuned binaries corresponding to different batch sizes of a model reside in the same container, as they belong to the same client model and need not be isolated.

Each container and the controller share two producer-consumer queues (respectively), which we use for fast inter-process communication. Each container, upon receiving an input request passes the request meta-data to the controller over their mutual producer-consumer queue. Additionally, each container pre-processes the input data on the CPU cores or specific hardware accelerator of the edge platform. The IceEdge controller is invoked periodically every 150 ms (this is configurable), and consumes new pending requests from all the incoming queues. It computes partial schedules for the pending requests and sends them to the respective containers over the respective producer-consumer queues. If no requests are pending, the controller goes back to sleep.

The containers do not run the inferences immediately upon receiving the partial schedules from the controller. The controller first receives completion notifications from each container for the requests that were issued to the GPU in the previous iteration. When the controller receives the last completion notification, it sends a *continuation* message to each container. Subsequently, each container sends the next batch of inferences on the GPU, as per the partial schedule computed by

the controller. This helps handle the effect of any unexpected GPU overruns, for instance when the inference time exceeds that predicted by the controller when coming up with the partial schedules. Specifically, it allows the containers to drop requests even at the last moment, if the prior request overran its duration, thereby avoiding cascading effects. This mechanism also allows IceEdge to ensure that the GPU is used in the expected concurrency mode, and that GPU overruns do not lead to scenarios where more than the desired number of requests are inferred concurrently on the GPU, which may further increase the variability in response times.

IceEdge implements the thermal control mechanism which ensures the temperature of the hardware platform is maintained within ambient level. IceEdge dynamically sets the GPU frequency to maintain the platform temperature while aiming to minimize the frame drop rate. Experiments performed at various frequency levels on Jetson AGX platform show the trade-off between GPU frequency, temperature, throughput, and GPU utilization (figure 1b & 2). In order to maximize performance, the IceEdge thermal controller maintains the frequency at $F_{turbo} = 1377MHz$, resulting in the best throughput. To control the sharp rise in temperature, the controller sets the frequency at $F_{safe} = 1032MHz$ to maintain the temperature performance trade-off (as observed in figure 6). This disables Linux DVFS and prevents it from interfering with IceEdge's thermal control. Every time IceEdge controller is invoked, it checks **(i)** if the total number of outstanding requests exceeds the threshold $T_{queue} = 12$ *(chosen based on the frame arrival rate & controller invocation period)* and **(ii)** whether the rise in the temperature in the last $M = 60$ seconds (400 scheduling iterations of the controller) T_{temp} is ≤ 5°C. If both (i) and (ii) hold, then the GPU clock frequency is tuned to F_{turbo}, while as if (i) continues to hold while (ii) does not (i.e., the temperature rises by greater than 5°C, the controller will tune the frequency to F_{safe} until the temperature falls by $T_{temp} = 5$°C again.

5 EVALUATION

We evaluate IceEdge by answering five main questions.
(a) Section 5.1: What is the overhead of containerizing DNN inference on edge platforms? Does each inference take more time in IceEdge, compared to when running on bare metal edge hardware, as in prior work [45]?
(b) Section 5.2: With containerization for application isolation, reduced CPU-GPU clock frequencies for thermal safety, and its novel scheduling approach, can IceEdge match the performance of state-of-the-art inference serving frameworks Triton [5] and DeepRT [45]? How does its scheduler design – which allows a concurrency factor of up to two and uses a deadline-aware scheduler with dynamic batching – help reduce frame drop rates?
(c) Section 5.3: How effective is IceEdge at populating its profiling table at runtime, and how does this help IceEdge come up with effective concurrency-aware schedules?
(d) Section 5.4: How well does IceEdge maintain platform temperature with appropriate choice of the CPU-GPU clock frequencies and other thermal control parameters, when compared with baselines Triton and DeepRT?
(e) Section 5.5: How well does IceEdge perform when DNN inference requests are generated based on a real traffic density trace, a realistic workload for edge platform?

We additionally provide a high-level overview of the existing designs and IceEdge to facilitate understanding of the comparisons in the forthcoming sections.
a) **Triton:** Executes models using either static or adaptive batching, with batch sizes configured by the user. It does not consider deadlines and does not drop frames.
b) **DeepRT:** Runs each model as a separate process with model-specific queues. It is deadline-aware and can batch requests across multiple instances of the same model.

Model Type	Task Type	Size	Bare metal	Docker
Resnet-18 [48]	Classification	44.8	20.12	20.48
Inception_v3 [44]	Classification	90.9	35.82	36.01
Resnet-50 [28]	Classification	105	48.09	48.32
TinyYolov3 [47]	Detection	38.9	24.38	24.45
Yolov2 [34]	Detection	195	92.94	93.07

Table 1. DNNs used in our evaluation study and their avg inference time (ms) for a batch size of 4.

c) **IceEdge:** Executes models with deadline awareness and supports concurrent execution of multiple models. It uses dynamic batching and may drop frames that cannot meet their deadlines.

5.1 Bare-Metal vs. Docker

We use multiple popular deep neural network (DNN) models from the literature for our evaluation study. These are listed in Table 1 along with their model sizes. These models are regularly used in edge inferences for two main classes containing model weights obtained from the model computer vision tasks - image classification and object detection. Each Docker container stores and executes inferences for one of these models, using one or more batch sizes as determined by the IceEdge controller. We derive the model implementations from model zoo [2] as an ONNX format, and as discussed in Section 4, tune them for different batch sizes (1, 4, 8, and 12) for our target NVIDIA Jetson AGX platform using TVM's utilities [3] for this process to generate the optimized models.

Table 1 also reports the average inference times when model inferences are executed on bare metal versus when executed inside a Docker container. In this experiment, we play a video workload with an input frame arrival rate of 30 frames per second (FPS) and an arrival pattern that follows Poisson distribution. This emulates a setup where the video frames are sent over the network directly from CCTV cameras, which are configured to capture videos at 30FPS, but the exact frame arrival times may not be strictly periodic because of networking and system overheads. We entirely bypass the IceEdge controller; instead, we consider a simple scheduling algorithm that waits for four frames to queue up and then executes a batch of four. Note that for both bare-metal and Docker scenarios, we use the same tuned model binaries and TVM for runtime inference. Results in Table 1 show that overheads incurred due to containerization are negligible. This observation holds also for other batch sizes.

5.2 IceEdge Latency Improvement

NVIDIA's Triton Inference Server [5] can be configured in two modes: (a) *static*, where a fixed number of requests are batched before being sent to the GPU. The controller waits indefinitely until enough requests have queued up to fill a batch. (b) *adaptive*, in which the controller batches requests until either a fixed batch size has been reached or until a configured time limit expires, whichever occurs first. In both modes, the batch size is passed as a configuration parameter by the user, and not determined dynamically or even automatically by Triton. In our experiments, we run Triton in the adaptive batching mode (as the static batching mode is clearly not ideal for soft real-time applications) with the batch size configured to 4 and the time limit configured to 150 ms (which is also the scheduling interval that we use for the IceEdge controller). For DeepRT, we maintain model specific queues instead of docker specific queues. Each NN model is run as a process and if there are multiple instances of the same model, DeepRT batches the requests as much as possible across them.

In our experiments, we vary the number of models from two to six. When using two models, we use the first two models in Table 1, i.e., Resnet-18 and Inception_v3; when using three models, we use the first three models in Table 1; and so on. When using six models, we use two separate

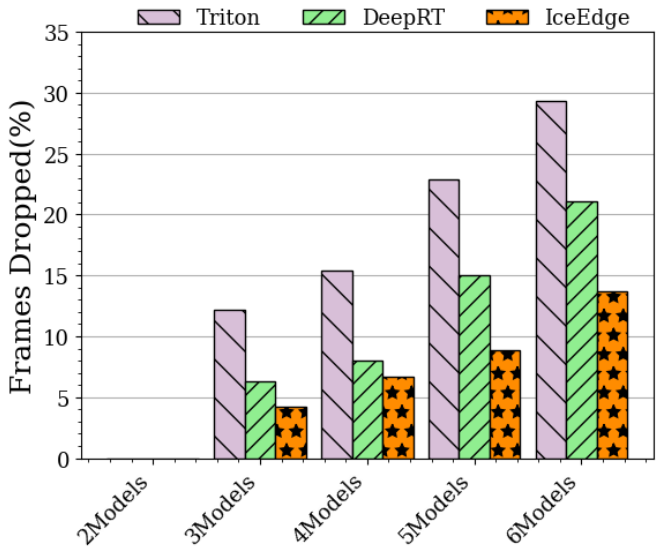


Fig. 4. Frame drop percentage in different frameworks for varying number of co-located models.

instances of the Resnet-18 model. We use a similar workload as discussed in Section 5.1 above, where request arrivals follow a Poisson distribution with an average arrival rate of 30FPS. The deadline for each of the inference requests for all the dockers is set to 300 ms.

IceEdge may drop frames if it believes they cannot be completed within their deadlines. Frames may be dropped either inside the IceEdge controller or even inside the Docker containers. Frames in IceEdge may occasionally overrun their expected inference times and complete after their deadlines. DeepRT, like IceEdge, is also deadline-driven. Triton, on the other hand, never drops frames, as it has no notion of deadlines, but may frequently end up executing frames after their deadlines. Hence, we use the "frame drop" percentage as our evaluation metric, which is the total number of frames dropped and that finished after their deadlines.

The experiment results are illustrated in Figure 4. They show the frame drop percentage for Triton, DeepRT, and IceEdge for five different configurations. In the first configuration, with two models, all inferences are completed within their respective deadlines for each baseline. However, as we increase the number of models from two to six, the workload becomes more difficult to schedule, and we observe that the frame drop percentage starts increasing for all baselines. Specifically, the frame drop percentage for Triton and DeepRT is on an average about 2x and 1.5x more than IceEdge across all configurations (respectively). Even for the second configuration (three models), which is relatively easier to schedule, Triton incurs a frame drop rate of around 12%, as opposed to 4% and 6% for IceEdge and DeepRT (respectively).

Triton's inferior performance is because of three reasons: **(i)** Triton runs TensorRT models whose execution times are reportedly more than TVM models [36]; **(ii)** Triton does not take request deadlines into consideration and in fact may routinely end up using the GPU for wasteful work (i.e., executing requests beyond their deadline); and **(iii)** Triton does not use any dynamic batching heuristics. In contrast, IceEdge is deadline-aware and uses an effective batching strategy that aims to maximize batching opportunities. In fact, we also compared IceEdge's batching strategy with

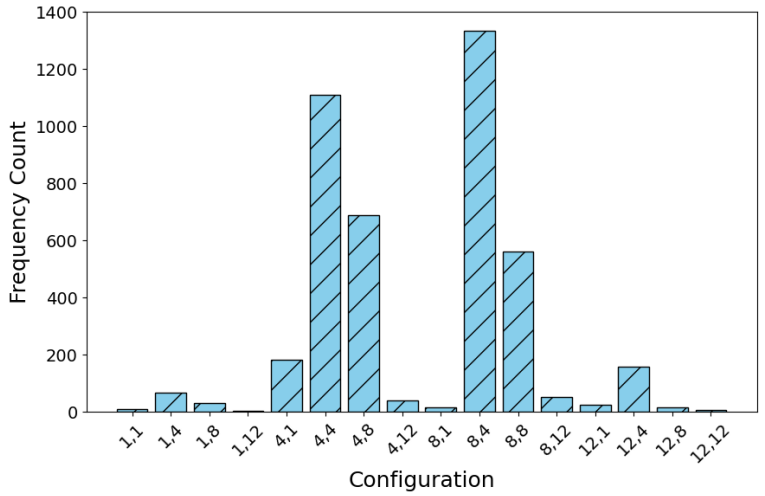


Fig. 5. Frequency count of the batch configurations for two models (Resnet-18 and Inception) when run concurrently.

that obtained by a constraint solver (in an offline mode), and observed that both report the same set of batch sizes.

While DeepRT is deadline-aware and performs better than Triton, it still incurs more deadline misses and frame drops than IceEdge. DeepRT completely gives up on concurrent execution in lieu of predictability; however, it also relies on deterministic request arrivals as well as deterministic pre-processing delays to benefit from its soft real-time design [45]. Since our experiment setup emulates a real environment where system overheads add noise to (seemingly periodic) arrival times, DeepRT's assumptions may not always hold. In this case, IceEdge's design allowing 2-way concurrency utilizes GPU capacity more efficiently, which in turn translates into better deadline compliance when there is a burst of requests. IceEdge hence also benefits more when we increase the number of models, as there are more opportunities for concurrent execution (e.g. in Figure 4, the gap between DeepRT and IceEdge increases from left to right).

5.3 Profiling Table Updates in IceEdge

Concurrently executing multiple instances of different models results in complex interference patterns between models. Prior work suggests that to design a soft real-time system, we need to first figure our the latency profile for all subsets of all admitted models. For instance, when there are n different models, we need to profile $\sum_{k=1}^{n} \binom{n}{k} = 2^n$ different combination, and even if we allow only 2-way concurrency, we need to profile $\binom{n}{2}$ different combinations, which is impossible [45]. In edge computing systems, n is typically small, and as shown earlier, 2-way concurrency is both sufficient and desired, from latency and throughput point of view. However, there still remains the problem of profiling $\binom{n}{2}$ different combinations upfront. As we show below, IceEdge is able to tackle this problem by bootstrapping the profiling table with inference times that are sum of the inference times of individual models, and then gradually, as more configurations are encountered over time, update the table with more accurate values.

We run IceEdge with two models (Resnet-18 and Inception_v3) for over 10k controller iterations (each iteration corresponds to the 150ms controller time period). We use the same configurations for IceEdge (e.g., 300 ms deadline and Poisson arrival at 30FPS) as in the previous section.

Con	I1+I2	2000	4000	6000	8000	10000	Diff
1,1	20.9	38.74	38.74	38.74	38.74	39.85	18.95
1,4	31.74	28.39	26.24	26.24	26.24	26.24	-5.50
1,8	60.13	58.23	58.23	58.23	58.23	58.23	-1.90
1,12	60.50	60.50	60.50	60.50	60.50	67.10	6.60
4,1	39.13	33.87	31.72	30.69	30.69	30.69	-8.44
4,4	49.97	44.87	42.86	38.21	36.39	33.19	-16.78
4,8	78.36	112.82	112.82	112.28	121.32	124.12	45.76
4,12	78.73	59.24	59.24	59.24	60.27	60.27	-18.46
8,1	65.52	90.19	90.19	90.19	90.19	90.95	25.43
8,4	76.36	68.23	64.91	62.32	61.08	58.66	-17.70
8,8	104.75	95.23	94.68	94.68	88.76	88.76	-15.99

Table 2. Profiling table updates of execution times (in ms) for IceEdge controller after every 2000 iterations.

The histogram in Figure 5 shows how IceEdge schedules each pair of batched model instances concurrently. For example, configuration (4, 1) implies the controller chose to infer a batch of four requests using Resnet-19 concurrently with a single inference of Inception_v3.

Table2 further reports latency profile in the profiling table for each configuration across these 10k iterations. The second column in the table represents the sequential execution of two inference tasks, which is simply $I1 + I2$. This value is also used to bootstrap the profiling table and hence indicates the latency profiling at initialization time. The subsequent columns represent the latency profiles after every 2000 iterations. The last column shows the difference in the latency between the final and the initial configurations. A negative value indicates that the latency profile has improved (and vice versa).

We make the following main observations. From Figure 5, we notice that only few configurations have relatively high frequencies, such as (4, 4), (4, 8), (8, 4), and (8, 8). We observe a similar trend with increasing number of models as well. From the last column in Table 2, we notice that most configurations benefit from concurrency as their latency profiles gradually improve as more measurements are incorporated. Although this is not true for all configurations. For configurations such as (1, 1) or (1, 12), which are hardly used by IceEdge's controller, the latency profiles remain volatile, and may improve over time. However, for configuration (4, 8), despite the frequency count being high, concurrent execution does not seem to yield any benefit. In this case, IceEdge will not perform concurrent execution with such configurations as it can result in potential deadline misses.

Overall, IceEdge's approach helps it to effectively circumvent the offline profiling problem identified in prior work.

5.4 IceEdge vs. Baselines: Thermal Safety

Experimental Setup For the different thermal safety parameters, we already discussed the details in Section 4. In addition, we use *tegrastats* [4] to obtain the temperature statistics when executing IceEdge or one of the baselines. We collect the temperature statistics for all systems, when provisioned with four models, for a duration of 30 minutes. Note that the ambient temperature is around 30°C for these experiments and we do not alter the fan settings in our setup, it remains off in the beginning and system automatically turns it on after a temperature rise is observed. This remains consistent across all the designs.

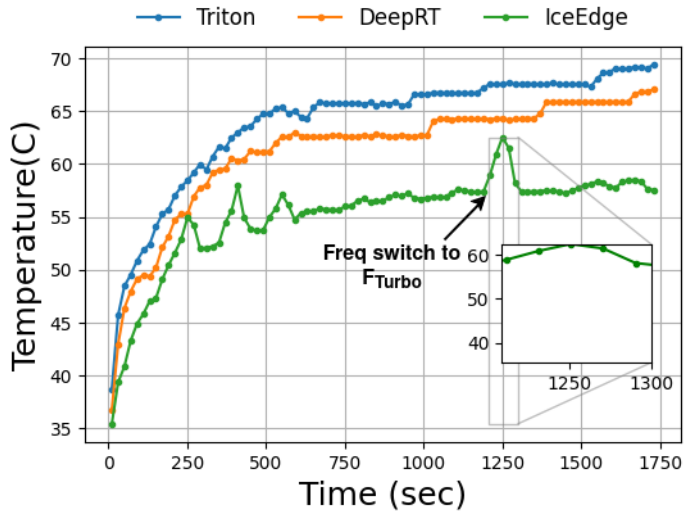


Fig. 6. Comparison of change in platform temperatures when running Triton, DeepRT, and IceEdge. IceEdge explicitly sets the GPU frequencies and switches based on the queue density and the rate of change of temperature.

Thermal Comparison Among the Designs Figure 6 illustrates their temperature profiles. We observe that the temperature rises in all systems with time, but the temperatures when running Triton and DeepRT are approximately 12°C and 10°C higher (respectively) compared to the temperatures when running IceEdge. Thermal aware edge Upon further investigation, we note that Linux DVFS sets the frequencies to the turbo mode in case of both Triton and DeepRT, leading to the rise in platform temperatures. However, IceEdge's controller explicitly controls the thermal aspect by auto-scaling GPU frequencies smartly with negligible effect on the application performance, as well as without causing the platform to overheat. EdgeEngine [6], a thermal aware Ml platform for edge devices is similar to IceEdge which also performs the GPU frequency scaling keeping in view the thermal aspect, but unlike IceEdge it does not provide concurrency. Also EdgeEngine only takes ambient temperature of platform into consideration while ignoring the effect of temperature increase during inference (continuous inferences). As depicted in Figure 10, IceEdge performs continuous inferences while maintaining the platform temperature. We observe from the Figure 6 (green line) that there are temperature peaks in between at some time intervals for IceEdge. This is because IceEdge keeps a check on the queue length and if it is greater than 12, it switches the frequency to F_{turbo}=1377MHz which leads to the rise in the temperature. When the rise becomes greater than 5°C, it switches the frequency back to F_{safe}=1032MHz leading to some dip in the temperature, thus ensuring the thermal safety.

5.5 Traffic Monitoring Case Study

We perform an additional case study using a traffic monitoring and density estimation dataset collected from a busy traffic intersection in a large metropolis. We obtain the dataset from Shafi et al. [35], who deployed traffic cameras at the intersection and collected 40 days of traffic density data. We use a subset of these video traces in our case study.

ML models used. High traffic density needs a more accurate object detection model like YOLOv2 to handle vehicle occlusion. However, since the visual scene changes less frequently when the traffic density is high, a low FPS may suffice in this case. Conversely, very high FPS is needed in

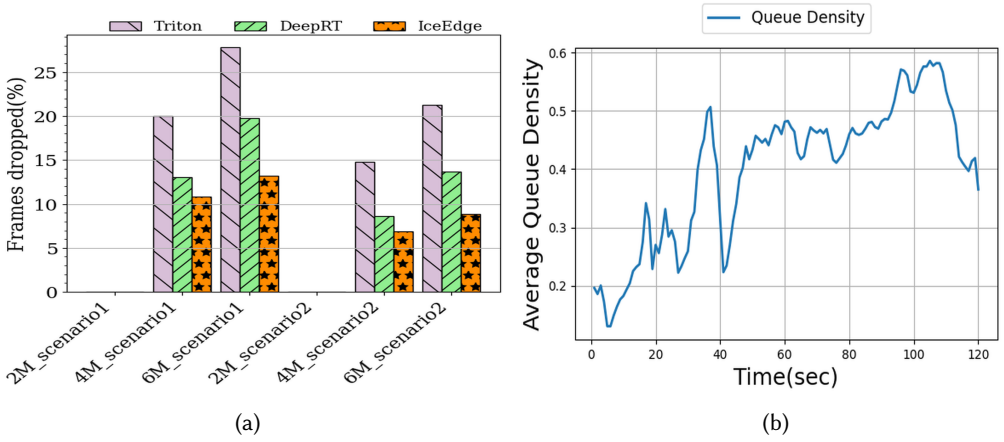


Fig. 7. (a) Frame drop percentage (b) 2 min trace from six cameras

low traffic density situations to catch high speed vehicles, but even a simple object detection model like TinyYOLO works well to detect the few vehicles in the empty road. Therefore, we create two IceEdge dockers, one for running the YOLOV2 object detection model and the other for running the TinyYOLO model. We additionally use 2-4 image classification models and instantiate a Docker for each of these. We added these models to simulate the effect of running few clients performing classification as well. In order to check the effectiveness of IceEdge, we added more models than just two detection models, because we did not observe any frame drop in just two model case, as can be observed in 7a.

Request arrival pattern. We evaluate the performance of IceEdge against Triton and DeepRT in this case study. The workload for TinyYOLO and YOLOv2 models is derived from the traffic density dataset, whereas the image classification models continue to have a Poisson arrival at 30 FPS.

For the object detection models, input image frames are tagged with traffic density values from the dataset. Using the density tag, the docker sets the necessary output FPS, which decides the deadline for that frame. We divide the density values into three bins, corresponding to 0.0-0.3, 0.3-0.6, and 0.6-1.0, respectively. We map each bins to a different output FPS. For higher densities, the FPS needed is less, so we map bin 0.6-1 to 10 FPS. Similarly, for lower densities, the FPS needed is more, so we map bin 0-0.3 to 30 FPS and bin 0.3-0.6 to an intermediate value of 20 FPS. Note that the bins will determine the FPS arrival based on Poisson and the effective deadline will the arrival time + deadline of 300 ms as set for our evaluation setup.

Camera to docker mapping. The dataset has traffic density values from 6 cameras at one intersection. Figure 7b shows the average density plot for a 40 days' data for 3 cameras each. We create two scenarios for these 6 cameras: (i) The two IceEdge dockers running TinyYolo and Yolov2 models take traffic density inputs from 3 cameras each. (ii) Yolov2 docker is chosen when the FPS needed for a camera is less (0.6-1 density bin) whereas TinyYolo docker is chosen when FPS needed is more (0.1-0.6 density bins. Note that we run these two dockers along with other classification models inside other dockers to see the real effect of deadline misses across the different schemes.

IceEdge performance comparison with baselines. We show the comparison between IceEdge and the two baselines, in terms of frames miss rate. Figure 7a shows the frame miss rate comparison for both scenarios. We observe from the Figure that the trends remain similar to what we found earlier in Section section 5.2. With only 2 models where we have TinyYolo and Yolov2 running for both scenarios, we observe there are no deadline misses for IceEdge and the baselines. However

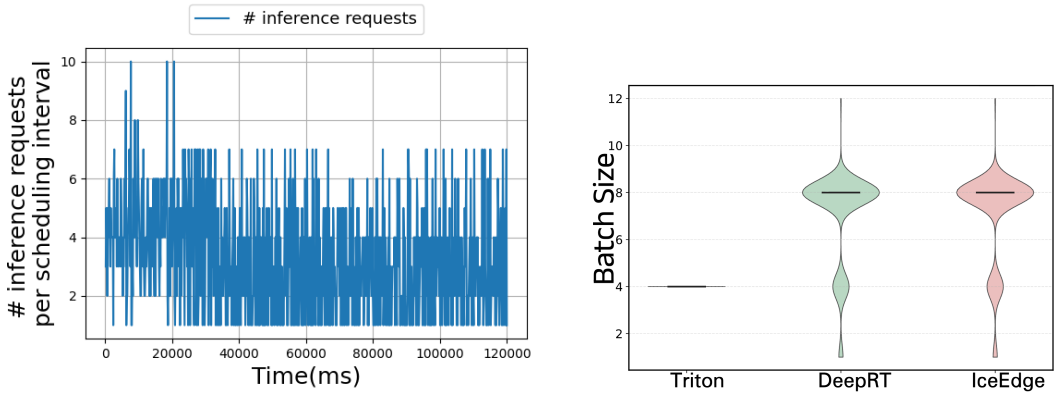


Fig. 8. (a) Inference request arrival pattern (b) TinyYolo batch size comparison.

as we increase the load from 2 to 4 models by adding two new classification models (Resnet-18 and Inception), and then from 4 to 6, we observe that for both scenarios, less frames miss their deadlines in IceEdge. Owing to the concurrency and dynamic batching along with scheduling using EDF, IceEdge has significantly less frame drop rates. We dig deeper into these results next.

Sample input request pattern. Figure 7b illustrates a two-minute traffic density trace averaged over 40 days of data i.e. the density value at a particular time of the day is averaged over 40 days. The density values range from 0.1 to 0.6, indicating the need for varying FPS requirements across different scenarios. These FPS requirements drive the frequency of inference requests for the dockers. As an illustrative example, Figure 8a shows the inference request arrival pattern for one of the dockers, TinyYolo, for this two-minute trace. In this context, we observe that lower density values (0.1-0.3) necessitate a higher FPS of 30, resulting in a greater number of inference requests within these ranges. Conversely, for density values between 0.3 and 0.6, the FPS requirement drops to 20, leading to fewer inference requests as depicted in Figure 8a. This pattern is a representative of the execution for the TinyYolo docker.

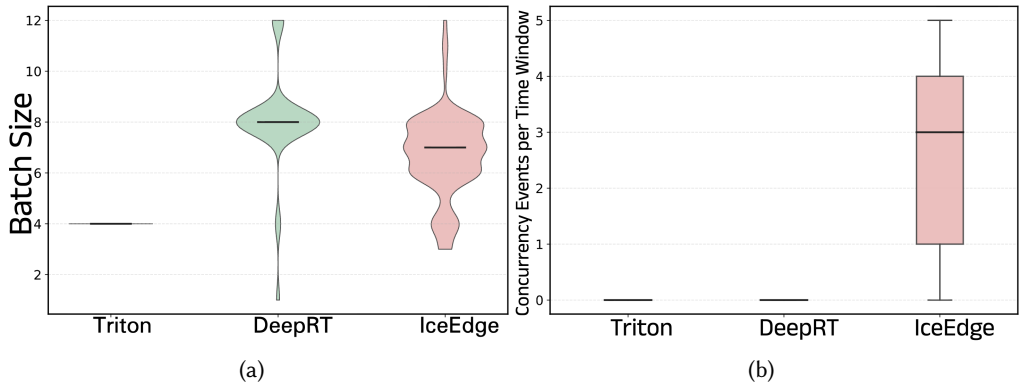


Fig. 9. (a) Batch size distributions for Resnet-18 across systems. (b) Concurrency comparison across Triton, DeepRT, and IceEdge

Batchsizes chosen by IceEdge vs. baselines. Given this input request pattern, Figure 8b presents the distribution of batch sizes for TinyYolo. Triton employs a fixed batch size configured at runtime, which we set to 4 in our experiments, resulting in a narrow distribution concentrated at that value. In contrast, IceEdge and DeepRT dynamically adjust batch sizes at runtime based on inference request deadlines, reflecting changes in traffic density. As shown in Figure 8b, this dynamic behavior leads to broader batch size distributions and higher median batch sizes for both IceEdge and DeepRT, with the two systems selecting similar batch sizes under comparable traffic conditions.

We also compare batch size behavior for Resnet-18 executed as two distinct applications. DeepRT considers all instances of Resnet-18 as a single entity, as a result it employs aggressive batching by clubbing all the requests without preserving process isolation between co-located applications. Figure 9a illustrates this behavior using a violin plot of batch size distributions. DeepRT exhibits a distribution skewed toward larger batch sizes, indicating that requests from the two Resnet-18 instances are batched together. In contrast, IceEdge design allows the process isolation thereby identifying the two Resnet-18 models as separate instances. In Figure 9a, IceEdge shows a broader but more controlled batch size distribution, reflecting dynamic batching decisions that allows application isolation. This design allows batching requests across various applications separately (even if different applications use identical model) maintaining application (process) isolation. This is particularly important in case of multi-tenancy setting running concurrent applications. In contrast Triton exhibits a narrow distribution, consistent with largely fixed batching behavior.

Concurrency chosen by IceEdge vs. baselines. We also check whether IceEdge utilizes concurrency. As discussed in Section 3, IceEdge supports limited concurrency with a maximum of 2 concurrent models, whereas Triton and DeepRT do not support concurrency at all. Figure 9b presents a box plot comparing concurrency behavior across Triton, DeepRT, and IceEdge over the full 20-minute execution. Concurrency is sampled every 150 ms and aggregated into fixed time windows; each box summarizes the distribution of the number of sampling instants within a window at which concurrency is observed. Triton and DeepRT exhibit zero concurrency throughout the experiment, resulting in distributions collapsed at zero in Figure 9b. In contrast, IceEdge shows a non-zero median and a wide interquartile range, indicating that concurrency is frequently enabled based on the inference request pattern (Figure 8a) but varies over time. The spread of IceEdge's distribution suggests that concurrency is opportunistic rather than constant, with intermittent bursts of higher activity.

In summary, our experiments highlight the superior performance of IceEdge in handling varying inference request patterns, using more suitable batch sizes and opportunistic concurrency, compared to the baselines. IceEdge's ability to dynamically adjust batch sizes based on real-time conditions ensures that it can handle higher inference loads with fewer frame drops. The concurrency support in IceEdge further enhances its throughput capabilities by allowing multiple models to run simultaneously when the system load permits, thereby optimizing resource utilization.

Thermal safety. We earlier observed that existing DVFS schemes which are application agnostic run at the turbo frequency leading to higher rise in temperature compared to IceEdge. Figure 10 shows the temperature rise over around 16 hours traffic density data averaged over 40 days' using IceEdge. We observe that our controller is able to maintain the temperature well below the thermal trip point (shown in red line) by around 15°C. Thus IceEdge gives higher performance than Triton, while ensuring thermal safety.

IceEdge performance with multiple cameras In the previous section, we compared the performance of IceEdge against baseline schemes using 6 cameras. In this section, we evaluate how increasing the number of cameras affects the performance of IceEdge relative to the baseline approaches. We simulate a high-demand scenario where each camera is required to deliver the

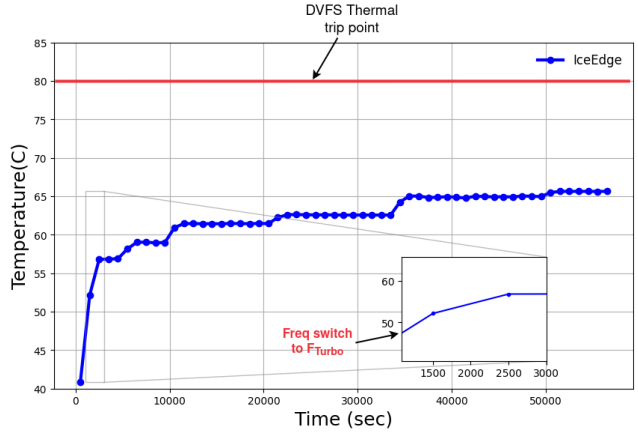


Fig. 10. Temperature rise when processing data with varying traffic densities averaged over 40 days. IceEdge-maintains the temperature well below the thermal trip point (80°C)

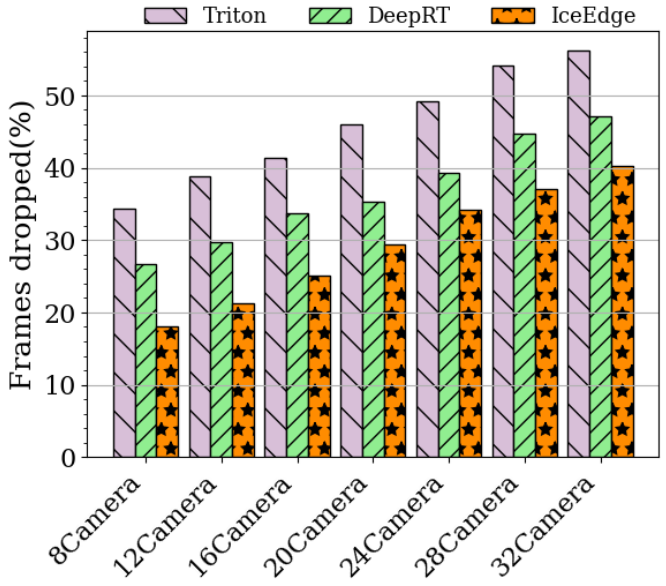


Fig. 11. Frame drop percentage comparison when we scale the number of cameras from 8 to 32. Each camera needs 30FPS as we assume the empty roads for this experiment

maximum possible FPS—30 FPS—under conditions of no road traffic. The number of cameras is incrementally increased from 8 to 32, and we analyze the resulting frame drop percentages across all tested approaches. In the scenario with 32 cameras, the system must handle a total of 960 FPS (32 x 30 FPS), with inference requests following a Poisson distribution. These frames are processed by the TinyYolo container within a Docker environment. This analysis is based on a scenario involving 5 models, where the remaining models are classification models as done earlier. The deadline for each of the inference requests is 300 ms as set in earlier experiments as well. Figure 11 illustrates how frame drops increase as the number of cameras, and consequently the FPS demand, rises.

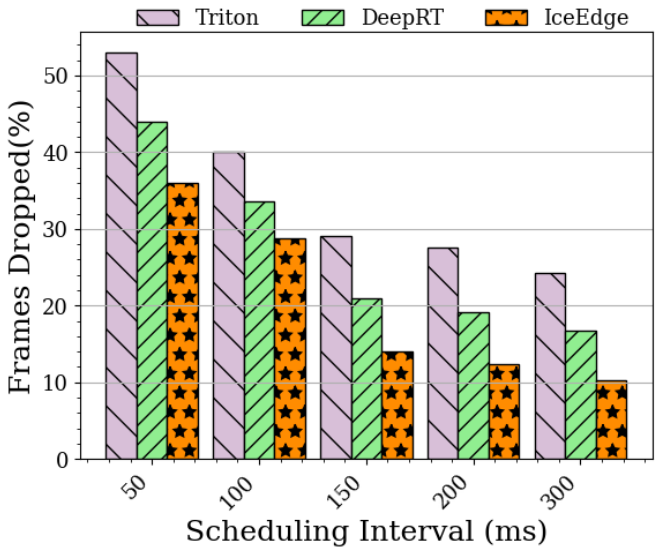


Fig. 12. Frame drop percentage in different frameworks for different scheduling intervals.

Notably, due to its concurrency, dynamic batching, and EDF-based scheduling, IceEdge consistently exhibits significantly lower frame drop rates compared to the baselines.

5.6 Sensitivity Analysis

Performance Comparision for Varying Scheduling Intervals In this section, we evaluate the performance of different designs against IceEdge under scheduling intervals of 50, 100, 150, 200, and 300 ms, as shown in Figure 12. With a low scheduling interval, requests are picked up sooner, but the batching window is limited. This results in smaller batches, reduced GPU utilization, and lower throughput. Since the system processes fewer frames within their deadlines, the number of dropped frames increases. In contrast, with a higher scheduling interval such as 150 ms or more, larger batches are formed, GPU utilization improves, and throughput increases. As a result, more frames are completed before their deadlines, and the frame drop rate decreases. *Note that we show the results for the representative execution of 6 models.*

Performance Comparison for Varying Deadlines In this section, we evaluate the performance of IceEdge compared to other designs under varying deadline constraints. Shorter deadlines result in more missed requests, as the system has less time to process each request, whereas longer deadlines allow more requests to be completed successfully, as shown in Figure 13. This behavior highlights the trade-off between system responsiveness and the ability to batch and process requests efficiently, with tighter deadlines putting greater pressure on the scheduling and processing mechanisms.

Performance Comparison for Varying Arrival Rates Next, we evaluate the performance of IceEdge compared to other designs under different arrival rates of 15, 30, 45, and 60 FPS (frames per second). Figure 14 presents the percentage of frames dropped for each design. At lower arrival rates, such as 15 FPS, very few frames are dropped. As the arrival rate increases, the number of dropped frames rises substantially, since all designs struggle to process the higher volume of incoming frames in real time. Notably, even at the highest arrival rates, IceEdge consistently outperforms the other designs, effectively handling more frames and reducing drops compared to existing approaches.

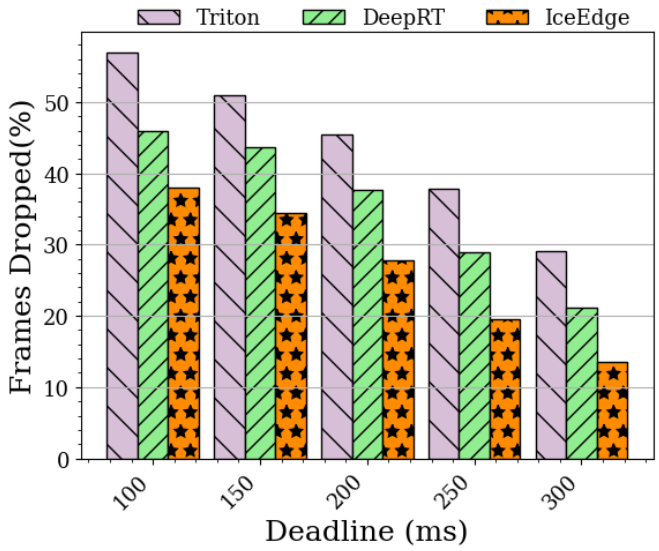


Fig. 13. Frame drop percentage in different frameworks for varying deadlines.

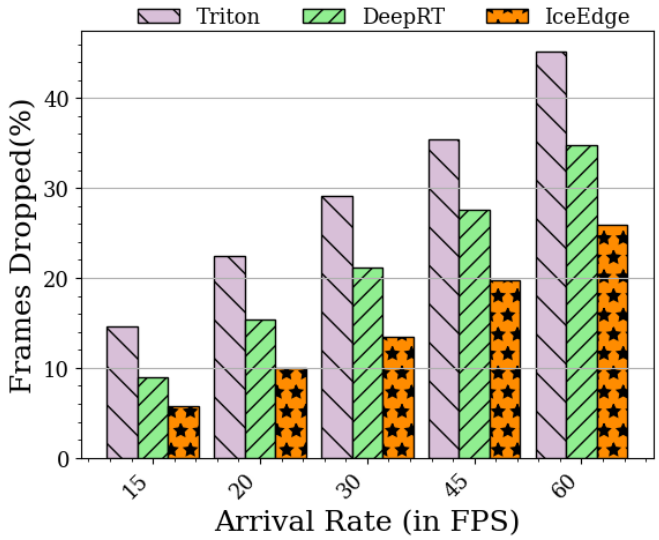


Fig. 14. Frame drop percentage with varying arrival rates

6 RELATED WORK

Inference Scheduling in Cloud. QosSched [19] uses deep reinforcement learning (RL) to adaptively choose ML model and inference batch size to balance inference latency vs. accuracy. Clipper [16] introduces caching, batching, and adaptive ML model selection techniques to improve latency and throughput, but it does not provide predictability for inference latencies. Clockwork [22] reduces arbitrarily long tail latencies for ML inferences, by planning tasks ahead of time, and disallowing concurrency. IceEdge is tailored for edge platforms and uses simpler scheduling heuristics than heavy computations in RL. Unlike Clockwork, we support limited concurrency of 2 improving

platform utilization.

Inference Scheduling on the Edge. NVIDIA's Triton Inference Server [5] does not support dynamic batching or provide latency guarantees, limiting its effectiveness for real-time applications that require predictable response times. DeepRT [45], like IceEdge, introduces dynamic batching and ensures latency guarantees using Earliest Deadline First (EDF) scheduling. However, IceEdge employs a more practical batching strategy tailored for multi-tenant environments by batching only requests from the same application. This design choice is driven by the fact that model weights typically differ across applications, making cross-application batching inefficient. Additionally, IceEdge implements limited concurrency, striking a balance between maximizing throughput and maintaining low latency, as empirically validated in our performance evaluations.

Existing works have explored different strategies for optimizing inference performance across heterogeneous platforms. Band [23] utilizes a variety of accelerators in mobile devices to execute different deep neural networks (DNNs) efficiently, whereas edge platforms predominantly rely on NVIDIA GPUs, requiring a different approach to workload management. Yi et al. [46] propose a fine-grained scheduling mechanism by decomposing neural networks into smaller, independently schedulable compute graphs. This approach offers greater control over resource allocation and execution timing. In contrast, IceEdge currently schedules entire neural networks as a single unit. If Docker containers were to partition their NNs into smaller subgraphs, IceEdge could seamlessly integrate this capability, though it would require maintaining larger profile tables to track execution characteristics.

Several other frameworks focus on optimizing inference efficiency through resource sharing and interference management. DeepQuery [18] enables the co-location of real-time (RT) and non-real-time workloads on shared GPUs, prioritizing RT tasks while striving to maximize non-RT task performance. Miriam [49] addresses contention in shared GPU environments by carefully orchestrating CUDA kernel execution to minimize intra-SM and inter-SM interference among co-located DNNs. μ-Serve [32]is a power-aware model-serving framework that combines fine-grained model multiplexing with dynamic GPU frequency scaling. Existing techniques such as model parallelism, placement, replication, and batching improve performance but do not exploit GPU frequency scaling. Despite these advancements, IceEdge differentiates itself by operating within separate Docker containers, where inference requests are strictly isolated. This design ensures process-level security and resource separation but inherently prevents simultaneous execution of inference requests within the shared GPU space due to process isolation constraints. By integrating practical batching policies, controlled concurrency, and tenant-aware scheduling, IceEdge presents a unique approach to optimizing inference performance in multi-tenant edge GPU environments.

Thermal safety at the edge. None of the papers listed above discuss thermal safety and leave it to Linux DVFS to reboot device or scale clock frequencies in case platform temperatures rise. EdgeEngine [6] is the framework designed with thermal aware capabilities for the edge and is the closest to our solution. It uses RL model to make scheduling decision in-order to maintain the platform temperature. Unlike IceEdge, EdgeEngine does consider concurrent model execution. Also it takes into account only two operating temperatures, i.e. 20°C & 70°C while missing out on the critical factor of looking into temperature rise due to model execution that can severely impact the system performance. The authors in [35] present an application dynamism and ambient temperature aware controller for Neural Network concurrency. It controls the number of threads and frequencies on the accelerator to manage application utility under strict thermal and power thresholds. However, it only focuses on traffic application as the use case and provides scheduling

decisions, without considering a mix of applications, each with possibly different FPS and frequency requirements.

Maity et al. [25, 26] address thermal management in heterogeneous MPSoCs for data-intensive Cyber-physical Systems using an analytical thermal model and Model Predictive Control (MPC). However, it might be impractical to profile and build analytical thermal models at different ambient temperatures, for the heterogeneous applications on shared GPU platforms. In contrast, IceEdge ensures thermal safety by a simpler, more practical approach of disabling DVFS and selecting appropriate operating frequencies at runtime, based on platform temperature and input workload. The work proposed by [39] introduce thermal-aware scheduling algorithms to optimize DNN execution on mobile devices by balancing performance and energy efficiency between GPU and NPU. While GPUs offer higher accuracy but cause overheating, NPUs are faster and more efficient but less accurate. The study introduces a Heuristic-Based Scheduling (HBS) algorithm that predicts temperature based on historical data and a Deep Reinforcement Learning Scheduling (DRLS) algorithm that optimizes long-term performance. However, it does not address the challenge of running multiple applications concurrently on the same hardware, as we do in IceEdge. Our work explores the complexities of resource contention and scheduling when multiple DNN workloads share the same GPU, ensuring fair resource allocation while maintaining optimal thermal and performance tradeoff.

7 CONCLUSION AND FUTURE WORK

In this paper, we present the challenges of deploying ML capabilities at the edge for smart city applications and address them with IceEdge, a novel framework for real-time inference on CPU-GPU based edge devices. IceEdge supports multi-tenancy, enhances inference performance compared to state-of-the-art methods, and controls platform temperatures. This allows efficient and safe consolidation of multiple applications on fewer devices, making it ideal for smart city deployment in regions with infrastructure constraints and extreme weather. To increase adoption, strong isolation mechanisms must be integrated without compromising performance, and schedules must adapt to diverse edge device types. Enabling these requirements will be the focus of our future work.

REFERENCES

[1] [n. d.]. Jetson Xavier AGX. https://www.nvidia.com/en-in/autonomous-machines/embedded-systems/jetson-agx-xavier/.
[2] [n. d.]. Jetson Zoo. https://elinux.org/Jetson_Zoo.
[3] [n. d.]. Python Autotune. https://tvm.apache.org/docs/how_to/tune_with_autotvm/tune_relay_cuda.html#sphx-glr-how-to-tune-with-autotvm-tune-relay-cuda-py.
[4] [n. d.]. Tegrastats. https://docs.nvidia.com/jetson/archives/l4t-archived/l4t-3231/index.html#page/Tegra%2520Linux%2520Driver%2520Package%2520Development%2520Guide%2FAppendixTegraStats.html%23wwpID0EJHA.
[5] [n. d.]. Triton Inference Server. https://developer.nvidia.com/triton-inference-server.
[6] Amirhossein Ahmadi, Hazem A Abdelhafez, Karthik Pattabiraman, and Matei Ripeanu. 2023. Edgeengine: A thermal-aware optimization framework for edge inference. In *Proceedings of the Eighth ACM/IEEE Symposium on Edge Computing*. 67–79.
[7] ARM Confidential Computing Architecture (CCA) [n. d.]. https://www.arm.com/architecture/security-features/arm-confidential-compute-architecture. Accessed on Jun 29, 2024.
[8] ARM TrustZone [n. d.]. https://www.arm.com/technologies/trustzone-for-cortex-m. Accessed on Jun 29, 2024.
[9] Ferdinand Brasser, David Gens, Patrick Jauernig, Ahmad-Reza Sadeghi, and Emmanuel Stapf. 2019. SANCTUARY: ARMing TrustZone with User-space Enclaves. In *NDSS*.
[10] Giorgio C Buttazzo. 2011. *Hard real-time computing systems: predictable scheduling algorithms and applications*. Vol. 24. Springer Science & Business Media.
[11] Sachin Chauhan, Kashish Bansal, and Rijurekha Sen. 2020. EcoLight: Intersection Control in Developing Regions Under Extreme Budget and Network Constraints. *34th Conference on Neural Information Processing Systems (NeurIPS 2020)*. https://proceedings.neurips.cc/paper/2020/file/97e49161287e7a4f9b745366e4f9431b-Paper.pdf
[12] Sachin Chauhan and Rijurekha Sen. 2023. RealLight: DRL based Intersection Control in Developing Countries without Traffic Simulators. In *NeurIPS 2023 Computational Sustainability: Promises and Pitfalls from Theory to Deployment*.

https://openreview.net/pdf?id=dmjT841VuV

[13] Sachin Chauhan and Rijurekha Sen. 2024. FrugalLight : Symmetry-Aware Cyclic Heterogeneous Intersection Control using Deep Reinforcement Learning with Model Compression, Distillation and Domain Knowledge. *ACM Journal on Computing and Sustainable Societies* (2024).

[14] Ling-Jyh Chen, Yao-Hua Ho, Hu-Cheng Lee, Hsuan-Cho Wu, Hao-Min Liu, Hsin-Hung Hsieh, Yu-Te Huang, and Shih-Chun Candice Lung. 2017. An open framework for participatory PM2. 5 monitoring in smart cities. *Ieee Access* 5 (2017), 14441–14454.

[15] Tianqi Chen, Thierry Moreau, Ziheng Jiang, Lianmin Zheng, Eddie Yan, Haichen Shen, Meghan Cowan, Leyuan Wang, Yuwei Hu, Luis Ceze, et al. 2018. {TVM}: An automated {End-to-End} optimizing compiler for deep learning. In *13th USENIX Symposium on Operating Systems Design and Implementation (OSDI 18)*. 578–594.

[16] Daniel Crankshaw, Xin Wang, Guilio Zhou, Michael J Franklin, Joseph E Gonzalez, and Ion Stoica. 2017. Clipper: A {Low-Latency} online prediction serving system. In *14th USENIX Symposium on Networked Systems Design and Implementation (NSDI 17)*. 613–627.

[17] Edge Computing Security: Risks, Considerations, and Best Practices [n. d.]. https://getstream.io/blog/edge-computing-security/. Accessed on Jun 29, 2024.

[18] Zhou Fang, Dezhi Hong, and Rajesh K Gupta. 2019. Serving deep neural networks at the cloud edge for vision applications on mobile platforms. In *Proceedings of the 10th ACM Multimedia Systems Conference*. 36–47.

[19] Zhou Fang, Tong Yu, Ole J Mengshoel, and Rajesh K Gupta. 2017. Qos-aware scheduling of heterogeneous servers for inference in deep neural networks. In *Proceedings of the 2017 ACM on Conference on Information and Knowledge Management*. 2067–2070.

[20] Ruben J Franklin et al. 2020. Traffic signal violation detection using artificial intelligence and deep learning. In *2020 5th international conference on communication and electronics systems (ICCES)*. IEEE, 839–844.

[21] Stephen Goldsmith. 2022. As the Chorus of Dumb City Advocates Increases, How Do We Define the Truly Smart City? *datasmart. ash. harvard. edu. Retrieved* 27 (2022).

[22] Arpan Gujarati, Reza Karimi, Safya Alzayat, Wei Hao, Antoine Kaufmann, Ymir Vigfusson, and Jonathan Mace. 2020. Serving {DNNs} like clockwork: Performance predictability from the bottom up. In *14th USENIX Symposium on Operating Systems Design and Implementation (OSDI 20)*. 443–462.

[23] Joo Seong Jeong, Jingyu Lee, Donghyun Kim, Changmin Jeon, Changjin Jeong, Youngki Lee, and Byung-Gon Chun. 2022. Band: coordinated multi-dnn inference on heterogeneous mobile processors. In *Proceedings of the 20th Annual International Conference on Mobile Systems, Applications and Services*. 235–247.

[24] Moritz Lipp, Daniel Gruss, Raphael Spreitzer, Clémentine Maurice, and Stefan Mangard. 2016. ARMageddon: Cache attacks on mobile devices. In *USENIX Security*.

[25] Srijeeta Maity, Anirban Ghose, Soumyajit Dey, and Swarnendu Biswas. 2021. Thermal-aware adaptive platform management for heterogeneous embedded systems. *ACM Transactions on Embedded Computing Systems (TECS)* 20, 5s (2021), 1–28.

[26] Srijeeta Maity, Rudrajyoti Roy, Anirban Majumder, Soumyajit Dey, and Ashish R Hota. 2022. Future aware dynamic thermal management in cpu-gpu embedded platforms. In *2022 IEEE Real-Time Systems Symposium (RTSS)*. IEEE, 396–408.

[27] Microsoft brings advanced hardware security to Server and Edge with Secured-Core [n. d.]. https://www.microsoft.com/en-us/security/blog/2021/03/02/microsoft-brings-advanced-hardware-security-to-server-and-edge-with-secured-core/. Accessed on Jun 29, 2024.

[28] Ishrat Zahan Mukti and Dipayan Biswas. 2019. Transfer learning based plant diseases detection using ResNet50. In *2019 4th International Conference on Electrical Information and Communication Technology (EICT)*. IEEE, 1–6.

[29] Nathan Otterness, Ming Yang, Tanya Amert, James Anderson, and F Donelson Smith. 2017. Inferring the scheduling policies of an embedded CUDA GPU. *OSPERT'17* (2017).

[30] Mohammad Khalid Pandit and Mahroosh Banday. 2022. Variance-Guided Structured Sparsity in Deep Neural Networks. *IEEE Transactions on Artificial Intelligence* 4, 6 (2022), 1714–1723.

[31] Mohammad Khalid Pandit, Roohie Naaz, and Mohammad Ahsan Chishti. 2021. Learning sparse neural networks using non-convex regularization. *IEEE Transactions on Emerging Topics in Computational Intelligence* 6, 2 (2021), 287–299.

[32] Haoran Qiu, Weichao Mao, Archit Patke, Shengkun Cui, Saurabh Jha, Chen Wang, Hubertus Franke, Zbigniew Kalbarczyk, Tamer Başar, and Ravishankar K Iyer. 2024. Power-aware deep learning model serving with {μ-Serve}. In *2024 USENIX Annual Technical Conference (USENIX ATC 24)*. 75–93.

[33] Francisco Romero, Qian Li, Neeraja J Yadwadkar, and Christos Kozyrakis. 2021. {INFaaS}: Automated model-less inference serving. In *2021 USENIX Annual Technical Conference (USENIX ATC 21)*. 397–411.

[34] Jun Sang, Zhongyuan Wu, Pei Guo, Haibo Hu, Hong Xiang, Qian Zhang, and Bin Cai. 2018. An improved YOLOv2 for vehicle detection. *Sensors* 18, 12 (2018), 4272.

[35] Omais Shafi, Sachin Kumar Chauhan, Gayathri Ananthanarayanan, and Rijurekha Sen. 2022. DynCNN: Application Dynamism and Ambient Temperature Aware Neural Network Scheduler in Edge Devices for Traffic Control. In *Proceedings of the 5th ACM SIGCAS/SIGCHI Conference on Computing and Sustainable Societies (COMPASS '22)*. Association for Computing Machinery, New York, NY, USA, 513–528. https://doi.org/10.1145/3530190.3534823

[36] Omais Shafi, Mohammad Khalid Pandit, Amarjeet Saini, Gayathri Ananthanarayanan, and Rijurekha Sen. 2024. Repercussions of Using DNN Compilers on Edge GPUs for Real Time and Safety Critical Systems: A Quantitative Audit. *ACM Journal on Emerging Technologies in Computing Systems* 20, 1 (2024), 1–25.

[37] Md Maruf Hossain Shuvo, Syed Kamrul Islam, Jianlin Cheng, and Bashir I Morshed. 2022. Efficient acceleration of deep learning inference on resource-constrained edge devices: A review. *Proc. IEEE* 111, 1 (2022), 42–91.

[38] Supraja Sridhara, Andrin Bertschi, Benedict Schlüter, Mark Kuhne, Fabio Aliberti, and Shweta Shinde. 2024. ACAI: Protecting Accelerator Execution with Arm Confidential Computing Architecture. (2024).

[39] Tianxiang Tan and Guohong Cao. 2024. Thermal-aware scheduling for deep learning on mobile devices with NPU. *IEEE Transactions on Mobile Computing* (2024).

[40] Chun-Ming Tsai, Jun-Wei Hsieh, Ming-Ching Chang, Guan-Lin He, Ping-Yang Chen, Wei-Tsung Chang, and Yi-Kuan Hsieh. 2023. Video analytics for detecting motorcyclist helmet rule violations. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*. 5365–5373.

[41] Chenxu Wang, Fengwei Zhang, Yunjie Deng, Kevin Leach, Jiannong Cao, Zhenyu Ning, Shoumeng Yan, and Zhengyu He. 2024. CAGE: Complementing Arm CCA with GPU Extensions. In *NDSS*.

[42] Hua Wei, Chacha Chen, Guanjie Zheng, Kan Wu, Vikash Gayah, Kai Xu, and Zhenhui Li. 2019. PressLight: Learning Max Pressure Control to Coordinate Traffic Signals in Arterial Network. In *Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining (KDD '19)*. 1290–1298.

[43] Hua Wei, Nan Xu, Huichu Zhang, Guanjie Zheng, Xinshi Zang, Chacha Chen, Weinan Zhang, Yamin Zhu, Kai Xu, and Zhenhui Li. 2019. CoLight: Learning Network-level Cooperation for Traffic Signal Control. In *Proceedings of the 28th ACM International Conference on Information and Knowledge Management (CIKM '19)*.

[44] Xiaoling Xia, Cui Xu, and Bing Nan. 2017. Inception-v3 for flower classification. In *2017 2nd International Conference on Image, Vision and Computing (ICIVC)*. IEEE, 783–787.

[45] Zhe Yang, Klara Nahrstedt, Hongpeng Guo, and Qian Zhou. 2021. Deeprt: A soft real time scheduler for computer vision applications on the edge. In *2021 IEEE/ACM Symposium on Edge Computing (SEC)*. IEEE, 271–284.

[46] Juheon Yi and Youngki Lee. 2020. Heimdall: mobile GPU coordination platform for augmented reality applications. In *Proceedings of the 26th Annual International Conference on Mobile Computing and Networking*. 1–14.

[47] Zhang Yi, Shen Yongliang, and Zhang Jun. 2019. An improved tiny-yolov3 pedestrian detection algorithm. *Optik* 183 (2019), 17–23.

[48] Xiang Yu and Shui-Hua Wang. 2019. Abnormality diagnosis in mammograms by transfer learning based on ResNet18. *Fundamenta Informaticae* 168, 2-4 (2019), 219–230.

[49] Zhihe Zhao, Neiwen Ling, Nan Guan, and Guoliang Xing. 2023. Miriam: Exploiting Elastic Kernels for Real-time Multi-DNN Inference on Edge GPU. *arXiv preprint arXiv:2307.04339* (2023).